

SPRINT: Structured Processing using Robust Integrated Neural Templates

Sebastian Blogg

University of South Wales

Table of Contents

1. Introduction 5
1.1. Background 5
1.2. Aims 6
1.3. Objectives 6
2. Literature Review 7
2.1. Historical Analysis of NLG/DA Techniques 7
2.1.1. Rule Based Systems 7
2.1.2. Corpus Based Systems 8
2.1.3. Probabilistic Techniques 8
2.1.4. Machine Learning 9
2.1.5. Neural and Autoregressive Modelling 9
2.1.6. Templatic Approach 9
2.2. Datasets and Usage 11
2.2.1. PubLayNet 11
2.2.2. DocBank 11
2.2.3. PubLaySharp 11
2.2.4. PubGenNet 12
2.2.5. Conclusion 12
3. Modern approaches to Document Automation 13
3.1. Introduction to LLMs 13
3.2. LLM Architectures 13
3.2.1. BERT 13
3.2.2. T5 13
3.2.3. GPT 14
3.2.4. LLaMA 14
3.3. Document Generation Models 15
3.3.1. LayoutGPT 15
3.3.2. DONUT 15
3.3.3. DocSynthV2 16
3.4. Literature Review Conclusion 16
3.4.1. Motivation 16
4. Methodology 18
4.1. Dataset Preparation 18
4.1.1. Training Data 18
4.1.2. Text extraction and Cleaning 18
4.1.3. JSONL Construction 19

4.2. Tokenization 19
4.2.1. Tokenizers 19
4.2.2. Prompt-Output Format 19
4.2.3. Tokenization and Loss Masking 20
4.2.4. Padding and Truncation 20
4.2.5. Batch Tokenization Efficiency 20
4.3. Fine-Tuning 21
4.3.1. LoRA Fine-Tuning with SHEARED LLaMA 21
4.3.2. Training configuration 21
4.4. Flask Web App 22
4.4.1. Web App 22
4.4.2. Local Inference First 22
4.4.3. Fallback to Hugging Face API 22
4.4.4. Prompt Template and Generation Setting 23
4.4.5. Input to Response to XML 23
4.4.6. XML Conversion to Document 23
4.5. Evaluation Metrics 24
4.5.1. XML Well-Formedness 24
4.5.2. BLEU Score 24
4.5.3. ROUGE Scores 25
5. Results 26
5.1. Data Preparation Results 26
5.1.1. Original JSON Files 26
5.1.2. Original Text Files 27
5.1.3. Matched Text XML Pairs 28
5.1.4. Pre-processed Data 28
5.1.5. Tokenized Data 28
5.1.6. Training Configurations 29
5.2. Validation Scores 30
5.3. GridSearch Optimisation 31
5.4. Evaluation Metrics 32
5.4.1. XML Wellformedness Scores 32
5.4.2. BLEU Scores 32
5.4.3. ROUGE Scores 33
5.5. Web App Results 34
6. Conclusion 37
6.1. Final Remarks 37

6.2. Future Work and Recommendations 39
Bibliography.....41
Table of Figures 43

1. Introduction

The need to streamline document handling processes has long been a cornerstone of operational efficiency across industries. As organizations have shifted from paper-based workflows to digital systems, document automation (DA) has emerged as a key area of focus, not just for improving speed and accuracy, but for reducing the cognitive and administrative burden placed on staff.

However, document automation remains a complex and often fragmented problem. While digitization has made documents easier to store and share, the automation of document generation itself continues to face major challenges, particularly in real-world contexts where input formats are inconsistent, client requirements are highly specific, and document structures vary across use cases. In many enterprise environments, documents must be generated from incomplete or unstructured data, adapted for different audiences, and maintained in compliance with strict formatting standards. This variety often exposes the limitations of existing tools, which tend to be rigid, narrowly scoped, or overly dependent on manually defined templates.

The promise of DA is to create structured, standardized documents with minimal human intervention. Yet the practical reality often involves extensive manual oversight, frequent reformatting, and bespoke adjustments that undermine scalability. Solutions that work for one client, team, or document type rarely generalize well to others. As organizations grow, this fragility becomes increasingly costly.

This project responds to that gap by exploring how structured document generation can be made more scalable, adaptive, and capable of handling diverse document types and content formats while maintaining structural accuracy. The approach focuses on creating a practical framework that can support real-world automation needs, with a particular emphasis on aligning loosely structured inputs to well-formed, standardized outputs.

1.1. Background

Document automation plays a crucial role in reducing repetitive workload, improving consistency, and accelerating information workflows across a broad range of sectors. Yet despite decades of innovation, organizations still face significant challenges when attempting to implement robust and scalable automation solutions, especially in environments where document types, content structures, and client requirements vary widely.

At its core, DA aims to transform raw, often unstructured, inputs into clearly formatted, semantically accurate documents. In practice, this involves interpreting user-generated content, applying appropriate structural rules, and producing outputs that meet internal or external standards. [1] While this may be straightforward in highly constrained domains, most business contexts are messier: documents change across projects; clients impose bespoke formatting constraints; and inputs are incomplete, inconsistently structured, or heavily contextual. This variety undermines the reliability of many current automation systems.

Traditional solutions such as rule-based engines or rigid template frameworks work well in environments where inputs and outputs are tightly controlled [2]. However, they break down when required to adapt dynamically. Adjusting these systems often involves costly reconfiguration or redevelopment. As organizations scale, the problem compounds: what worked for a single team or client quickly becomes inadequate in a broader operational context.

This is a challenge frequently encountered by Amber Labs in its consultancy work. Clients often request structured outputs that must align with specific institutional formats, compliance standards, or user-defined schemas. Yet these same clients rarely supply clean, consistent data. Documents must be reshaped on the fly; converted, validated, and formatted while preserving both meaning and layout integrity. The result is a heavy reliance on manual intervention, despite the theoretical promise of automation.

What's needed is a more flexible, adaptable approach to document generation, one that can generalize across document types, respond to varying content structures, and ensure outputs remain coherent, standardized, and structurally well-formed. This project addresses that need by exploring how structured document generation can be made more scalable and resilient in practice, not just in theory.

Recent advances in data science, machine learning, and natural language processing have opened new pathways for tackling this challenge. By leveraging the expressive capabilities of language-based models, there is growing potential to automate document generation not just at the formatting level, but at the level of content understanding, layout structure, and semantic alignment. These innovations suggest a promising frontier for reimagining how documents are synthesized from unstructured or semi-structured inputs.

To ground this work in a robust understanding of what has come before, this project begins with an investigation into approaches to document automation and natural language generation. The literature review that follows forms the foundation for designing a practical, scalable framework for structured document generation.

1.2. Aims

The aim of this research is to design, build, and evaluate a proof-of-concept framework capable of supporting scalable, adaptable, and structured document generation. This includes implementing the framework within a representative document context to demonstrate its practical applicability.

1.3. Objectives

- Review existing literature and historical techniques in document automation and structured content generation.
- Develop a conceptual and technical blueprint for structured document generation.
- Identify and apply the proposed methodology to a representative problem space where document variation and structural integrity are both critical.
- Build a small-scale proof-of-concept system that demonstrates end-to-end transformation of raw or semi-structured input into well-formed, structured document outputs.
- Define and apply both quantitative and qualitative metrics to evaluate output quality, accuracy, and structural validity.
- Develop a lightweight inference application to showcase the framework in a user-facing environment.

2. Literature Review

This research explores the historical evolution and state-of-the-art techniques in Natural Language Generation (NLG) and document automation. We address three primary research questions:

1. What are the key approaches for automated document generation that enable adaptability and contextual responsiveness in diverse scenarios?
2. How can AI systems improve document generation by integrating contextual understanding, continuity, and adaptability to dynamic user needs?
3. What are the primary challenges in achieving accuracy, robustness, and flexibility in AI-driven document generation, and how can principles inspired by cognitive psychology address these challenges?

2.1. Historical Analysis of NLG/DA Techniques

2.1.1. Rule Based Systems

Natural Language Generation (NLG) and Document Automation (DA) have long been essential tools for improving workplace efficiency by streamlining the creation and handling of information. Early efforts focused on rule-based methodologies, which applied grammatical and syntactic rules to generate text from structured data. These systems introduced significant efficiencies by automating repetitive tasks, reducing human error, and ensuring consistency in document production.

Goldberg's Forecast Generator (FOG) system [1] serves as a seminal example of early NLG systems. Using a combination of predefined linguistic templates and placeholders, FOG transformed meteorological data into coherent and domain-specific forecasts.

The system leveraged Meaning-Text Theory (MTT) [3], which separated the semantic representation of information from its syntactic realization, ensuring grammatical and contextually appropriate output. Introduced by Melcuk, MTT is key in aiding the description of Language, referring in particular to the linguistic idea of "sprachlich" which puts specific emphasis on how meanings can be conveyed and interpreted through language choice.

Compared to previous approaches that were based around using large amounts of tabular data, systems like GoldBerg's FOG resulted in efficiency gains and an increase in the throughput of reporting. Allowing for a higher quantity of forecasting reports to be generated with an increase in workload or staff numbers.

The same can be said for AlethGen [4], a component for text-realisation/NLG that was applied to Weather Forecasting and Machine Translation. Noting similar performance gains in speed of completing. Taking less than 2 minutes compared to the upwards of 5 minutes per task for human writers.

FOG and AlethGen faced notable challenges, particularly in scalability. The rule-based approach struggled to accommodate multilingual contexts, as evidenced by the gendered syntactic and morphological complexities of languages like French. Adapting the system to new languages or domains often required extensive re-engineering, underscoring the rigidity of the framework. Goldberg acknowledged these limitations, noting that while such systems provided critical improvements over manual processes, they lacked the flexibility required for broader applications.

The rule-based paradigm laid the foundation for more sophisticated approaches but revealed an enduring need for adaptability and scalable qualities that subsequent methodologies sought to address.

While rule-based systems introduced structure and consistency, their limitations in adaptability necessitated the development of corpus-based methods, which leveraged real-world data to improve contextual accuracy.

2.1.2.Corpus Based Systems

The emergence of corpus-based approaches addressed many of the scalability limitations of rule-based systems. Reiter and Dale [2] championed this transition by introducing a methodology that leveraged text corpora to derive linguistic rules and conventions from real-world examples. This approach involved curating corpora to cover expected output variations, which were then refined into target text corpora to resolve inconsistencies and integrate domain-specific conventions.

PlanDOC, developed by McKeown et al. [5] was created to automate the documentation of telephone engineering activities for the company Bellcore. A corpus was created through the analysis of narratives written by the engineers. Allowing for the domain specific sublanguage, linguistic structure and variety of messages required for documentation. By leveraging the collection of insights, PlanDOC remained relevant and avoided being redundant.

PlanDOC in comparison to FOG highlights some of the advantages over its predecessor technique. Unlike rule-based systems, corpus-based methods allowed for greater adaptability without requiring explicit programming of rules. However, the approach was not without its challenges, particularly in acquiring adequate corpora in domains constrained by confidentiality or data scarcity.

Additionally, the complexity of knowledge acquisition determining what information was necessary and how it should be structured which highlighted the inherent difficulty of ensuring both contextual accuracy and domain relevance. Corpus-Based methods improved on Rule-Based but still lacked in areas of Scalability and Flexibility that would be addressed by later approaches.

2.1.3.Probabilistic Techniques

Probabilistic methods marked a significant shift toward more scalable and flexible NLG systems. Researchers like Belz [6] introduced probabilistic frameworks that employed statistical models to inform sentence planning and text generation. By learning generation probabilities from extensive corpora, probabilistic systems addressed the inherent variability of language while enabling data-driven decision-making.

Frameworks such as Probabilistic Context-Free Representational Underspecification (pCRU) epitomize this paradigm, combining linguistic theories with statistical models to streamline content determination and syntactic structuring. These methods reduced reliance on manual rule creation, offering enhanced scalability and adaptability. While Corpus methods relied on specific sub language analysis to perform in each domain, given enough data a probabilistic model could generalise better at capturing the trends in the same corpus without the extensive analysis.

However, probabilistic systems faced their own set of challenges, including the need for large, annotated datasets and potential interpretability issues, particularly when balancing linguistic coherence with statistical variability.

2.1.4.Machine Learning

Machine learning (ML) built on probabilistic approaches by leveraging advances in neural networks to automate complex decision-making processes. Early systems, such as those proposed by Thomson et al. [7], used neural architectures to optimize document planning and content structuring, learning from large, annotated datasets to generalize across diverse domains. This integration of ML techniques allowed for greater adaptability and scalability, overcoming many of the limitations of corpus-based systems in handling diverse contexts albeit at the cost of increased computational requirements and challenges in interpretability. Despite these hurdles, ML-based methods have become foundational in modern natural language generation (NLG), offering unparalleled flexibility and performance.

Deep neural networks have since been applied to learn mappings from inputs to outputs without manually defined rules. In contrast to traditional sequence–to–sequence models that convert an input sequence into a fixed–dimensional vector, LayoutGAN [8] directly generates structured document layouts. In LayoutGAN, the generator begins with a set of randomly positioned elements and refines their labels and geometric parameters using self–attention modules to capture spatial and semantic relationships. The model incorporates a differentiable wireframe rendering layer that converts layout parameters into wireframe images, enabling a convolutional discriminator to assess spatial properties such as alignment and occlusion. This method addresses the limitations of fixed–length representations and provides a framework for generating complex and coherent document layouts.

2.1.5.Neural and Autoregressive Modelling

The advent of neural and autoregressive modelling brought about a paradigm shift in document generation. Models like GPT [9] and BERT [10], powered by transformers, demonstrated unprecedented capabilities in generating fluent and coherent text. Autoregressive models, such as those used in DocSynthv2 [11], extended these advancements by integrating layout and content generation within a unified framework.

These models excelled in capturing complex language patterns and maintaining contextual coherence, enabling the generation of high-quality documents across diverse applications. However, their reliance on vast training datasets and computational resources highlighted the trade-offs between performance and accessibility. Moreover, ensuring factual accuracy and logical consistency remains an ongoing challenge, particularly in specialized domains where precision is critical.

2.1.6.Templatic Approach

Template-based approaches emerged as a pragmatic solution for natural language generation (NLG) by directly mapping non-linguistic inputs into linguistic outputs using predefined fill-in-the-gap templates. Early systems, such as Goldberg's Forecast Generator [1], employed fixed templates to generate coherent forecasts from structured data, thereby achieving significant efficiency gains over manual methods. Although these systems were initially criticized for their rigidity and limited adaptability, later research by van Deemter, Krahmer, and Theune [12] challenged this view, demonstrating that the distinction between template-based and so-called "standard" NLG systems is increasingly blurred. Modern template-based systems have evolved to incorporate sophisticated linguistic rules, including recursive gap-filling and context-sensitive variations, which enable them to produce output that is both flexible and linguistically well-founded.

In the context of document automation, template-based approaches offer distinct advantages that make them a natural choice for generating standardized, structured documents. Their ability to directly map structured data into coherent text is particularly

beneficial for applications such as administrative reporting, legal document generation, and other domains where consistency and precision are critical. Systems like GOALGETTER exemplify this capability by converting structured inputs—such as tables of sports statistics—into natural language reports with high reliability. By leveraging these robust template mechanisms, our research adopts template-based methodologies as a core paradigm for document automation, aiming to achieve scalable and adaptable generation of structured documents while maintaining high linguistic quality.

2.2. Datasets and Usage

In the field of document generation, data serves as both the foundation and a significant challenge for advancing research. Modern Machine Learning (ML) and Generative Artificial Intelligence (Gen AI) rely heavily on high-quality datasets to train robust models. However, the limited availability accessible of certain datasets can hinder experimentation and Research. This section highlights key datasets that have influenced the domain of document analysis and generation, evaluated their applicability and limitations, and situated them in the context of ongoing research.

2.2.1.PubLayNet

Introduced by Zhong et al., PubLayNet [13] is a large-scale dataset developed by automatically aligning XML representations with the content of over 1 million PDF articles from PubMed Central. This dataset comprises over 360,000 document images, each annotated with five distinct layout elements: text, title, list, figure, and table. The creation of PubLayNet has enabled the training of deep neural networks that accurately recognize the layout of scientific articles. Moreover, their researched pre-trained models on PubLayNet have proven to be effective bases for transfer learning in different document domains, facilitating advancements in document layout analysis.

This diverse dataset enabled the training of deep neural networks—such as Faster-RCNN and Mask-RCNN—that achieved high mean average precision (MAP) scores in recognizing complex layout elements including text, titles, lists, figures, and tables. Furthermore, the pre-trained models on PubLayNet were shown to serve as strong initializations for fine-tuning on domain-specific datasets, such as health insurance Summary Plan Description (SPD) documents, where they outperformed models pre-trained on general image datasets like COCO and ImageNet.

2.2.2.DocBank

DocBank is a benchmark dataset developed by Li et al. [10] that comprises 500,000 document pages, each annotated at a fine-grained token level. The dataset is constructed through a weak supervision approach that leverages LaTeX source files from arXiv papers. In this process, the authors modify the LaTeX source , for example, inserting structure-specific colour commands to mark semantic units such as abstract, author, caption, equation, and title so that when the documents are compiled into PDFs, these visual cues directly correspond to distinct semantic roles. Advanced PDF parsing tools are then used to extract tokens and their bounding boxes, mapping them to the predefined labels based on the colour-to-structure correspondence.

This automated annotation strategy not only minimizes the labour-intensive effort typically required for manual Labelling but also produces a large-scale, rich dataset that supports both text-based sequence Labelling and image-based object detection methods. As a result, DocBank enables fair and comprehensive comparisons between models operating in different modalities, fostering the development of multi-modal approaches for document layout analysis. The dataset's public availability [14]further ensures that it serves as a valuable resource for advancing research in document automation and structured document generation.

2.2.3.PubLaySharp

PubLaySharp [15] is an open-source extension of PubLayNet, wherein the dataset has been converted into XML format. This conversion enables the training of models that can interpret

the relationship between content and layout within an XML framework, which is beneficial for applications requiring structured document representations. There are no papers which currently discuss using this extension of original dataset introduced by Zhong, opening a space to investigate the impact of using the XML representation alongside the original dataset of textual content and annotated images. While there is no research that has used this extension, it offers an opportunity to incorporate another medium of input into training.

2.2.4.PubGenNet

The research completed by Biswas et al. [11] was part of the motivation for this work, it would have been productive to be able to attempt to extend the capabilities of DocSynthV2, however both the dataset and the model itself are closed source. In contrast to the previously mentioned datasets, PubGenNet is not publicly available. PubGenNet is a curated benchmark tailored for document generation and completion tasks, instrumental in training models like DocSynthv2 for comprehensive document generation encompassing both layout and content. The unavailability of PubGenNet poses challenges for researchers seeking to replicate or build upon this work, highlighting the importance of accessible datasets in advancing the clearly active field.

2.2.5.Conclusion

Collectively, these datasets have been successfully utilized by prior research to drive advancements in document layout analysis. Their diverse modalities and rich annotations address many of the challenges inherent in document generation tasks. By leveraging these resources, our research can build upon established benchmarks and methodologies—enabling us to refine models for structured output generation, evaluate their performance comprehensively, and ultimately contribute to the development of more robust, adaptable document automation systems.

3. Modern approaches to Document Automation

3.1. Introduction to LLMs

Large Language Models (LLMs) represent a significant evolution in Document Automation. The scale at which text and, therefore documents has grown rapidly since the introduction of large language models. Which have been building on the transformer architecture introduced by Vaswani Et al. [16], in the paper 'Attention is All You Need'. This architecture uses attention mechanisms to prioritize relationships between input tokens, allowing models to capture long-range dependencies in text. Unlike previous recurrent or convolutional architectures, transformers enable parallel processing and better scalability, forming the backbone of modern LLMs.

The predominant approach employed by LLMs is autoregressive modelling, where the model predicts the next token in a sequence based on preceding tokens. This iterative process allows the model to generate coherent and contextually relevant outputs. Models like GPT-4 and its successors demonstrate how this methodology can produce nuanced text with remarkable fluency and precision. While primarily designed for natural language generation, LLMs have found applications in document automation. This versatility not only streamlines administrative and operational tasks but also opens up new possibilities for real-time document management and decision support, thereby underscoring the profound practical implications that motivate our research in this field that are still open to exploration.

3.2. LLM Architectures

The standard Large Language Model that handles text can be broken down into two sub structures of the encoder and decoder stack. Performing the attention mechanism using multiple attention heads simultaneously before passing this to the feed forward network to ultimately create probabilities on which tokens will come next within the body of text being generated. However, there are distinctions that should be discussed as this will help highlight why a specific architecture was employed for the research.

3.2.1.BERT

BERT, introduced by Devlin et al. [10], employs a bidirectional transformer architecture that enables contextual understanding of language by considering both preceding and succeeding words in a sequence during pre-training. Its masked language modelling (MLM) approach allows it to capture rich contextual representations. Subsequent models, such as RoBERTa by Liu et al. [17], optimized training by increasing data and computation and removing tasks like Next Sentence Prediction, enhancing performance across benchmarks. DistilBERT, developed by Sanh et al. [18], focused on reducing the model's size and computational demands through knowledge distillation while maintaining most of its accuracy. While these models were among the earliest examples of transformer architecture, they are not designed for generative tasks and, as such, are not applicable to this area of research.

3.2.2.T5

T5, or Text-to-Text Transfer Transformer introduced by Raffel [19], represents a significant leap in the unification of natural language processing tasks by reformulating all problems into a text-to-text framework. Its encoderdecoder architecture enables the model to treat every task—from translation and summarization to classification—as a sequence generation problem. This unified approach simplifies both the training process and downstream task adaptation, as the same model and tokenizer can be employed regardless of the specific application. By leveraging vast amounts of pre-training data and a span corruption objective

during pre-training, T5 is able to capture rich linguistic representations that facilitate high-quality, contextually aware text generation even in complex scenarios.

Building on T5's inherent flexibility, the work by Senadeera and Ive [20] introduces an extended soft prompt tuning method that incorporates learnable prompts at both the encoder and decoder levels. Their approach not only finetunes a small fraction of the model's parameters but also introduces a steering mechanism to guide the generation process, leading to outputs that adhere more closely to desired structural and stylistic constraints. This methodology is particularly relevant to our research in document automation, where generating wellformed and consistent structured outputs—such as XML —is critical. By integrating decoder-level soft prompts, their work informs our strategy to control and enforce output structure, suggesting that a similar technique could significantly improve the reliability and accuracy of automatically generated document formats.

3.2.3.GPT

The GPT series has evolved significantly since its inception, introducing advanced capabilities in each iteration. GPT-1 [9] laid the groundwork with its foundational pre-training and fine-tuning methodology. Building upon this, GPT-2 [21] scaled parameters substantially, enhancing open-ended text generation. GPT-3 [22] introduced few-shot learning, enabling generalization across diverse tasks without task-specific fine-tuning. GPT-4 [23] further advanced the model by incorporating multimodal capabilities, improved reasoning, and alignment strategies such as Reinforcement Learning from Human Feedback (RLHF) [24] to address limitations like factual inconsistencies.

However, it's important to note that the later models in the GPT series, specifically GPT-3 and GPT-4, are not open-source. OpenAI has not publicly released the source code or pretrained weights for these models. Instead, access is provided through OpenAI's API, allowing developers to integrate GPT-3 and GPT-4 functionalities into their applications.

This approach offers certain advantages, such as ensuring controlled usage and facilitating the implementation of safety measures. However, it also imposes limitations for researchers and developers who seek to understand the underlying mechanisms of these models or wish to fine-tune them for specific applications. The reliance on API access can restrict the flexibility needed for in-depth research and customization.

3.2.4.LLaMA

The LLaMA series introduces several architectural innovations that make it particularly promising for document generation tasks. One key quality is its ability to handle extended contexts: later iterations like LLaMA 3 [25] and 3.1 boast context window lengths of up to 128,000 tokens, which is especially beneficial for modelling lengthy documents or complex layouts where capturing intricate dependencies is critical. Moreover, techniques such as Rotary Positional Embeddings (RoPE) and Grouped Query Attention (GQA) enhance the model's capacity to encode hierarchical and sequential information. This structural awareness is essential for generating structured outputs—such as XML or other document formats—that require precise ordering and spatial coherence of elements.

Additionally, the series' efficiency improvements mean that even with a relatively smaller parameter count (as seen in LLaMA 1 [26] and LLaMA 2 [27]), these models achieve competitive performance compared to larger, less specialized systems. This balance of scalability and efficiency can be leveraged to develop document automation systems where resources may be limited, yet accuracy and consistency in generating both text and layout

are paramount. However, while the LLaMA series provides these promising features, its licensing restrictions—despite the public availability of weights—must be taken into account for high-scale or commercial document generation applications. Overall, the LLaMA series' extended context handling, refined attention mechanisms, and efficient parameter usage make it a viable candidate for transforming unstructured inputs into structured document outputs in NLG/DA research.

3.3. Document Generation Models

Document Generation Models are a series of models that offer some form of generation that is often bespoke to a specific task. They are constrained and narrow but often fit for purpose given task understanding.

3.3.1.LayoutGPT

Compositional Visual Planning and Generation with Large Language Models, by Feng [26] proposes a training-free, prompt-based framework that leverages large language models (LLMs) to serve as visual planners. Instead of requiring detailed manual inputs, LayoutGPT converts simple text conditions into structured CSS-like programs that describe 2D or even 3D layouts. The paper demonstrates that by composing in-context visual demonstrations—using familiar code-like representations—LLMs can generate plausible layouts that capture numerical counts, spatial relations, and even object attributes. This approach exploits the inherent reasoning and code-generation skills of LLMs such as GPT-3.5/4 to achieve strong performance on benchmarks that measure numerical and spatial reasoning. For instance, the authors show that LayoutGPT can accurately convert challenging textual descriptions (such as "there are three elephants standing beside a pool of water") into a detailed layout, which then serves as a reliable intermediate representation for downstream image or scene synthesis.

For our research in document automation, LayoutGPT offers a compelling proof of concept that LLMs can be "instructed" to generate structured layouts purely from natural language inputs. In this research, which explores automated document generation and the production of structured outputs like XML, LayoutGPT's CSS-style prompt methodology highlights an effective way to incorporate spatial and numerical reasoning into a generative pipeline. By drawing inspiration from LayoutGPT's approach to visual planning, we can envision adapting similar in-context demonstration techniques to guide the generation of document layouts that seamlessly integrate textual and formatting cues. This connection supports our goal of enhancing both the fidelity and interpretability of generated documents while reducing manual design burdens.

3.3.2.DONUT

Donut [27] represents a significant shift away from traditional OCR-dependent methods by employing an end-to-end encoder-decoder transformer for document understanding. Rather than relying on separate OCR modules to extract textual content, Donut directly processes document images and generates the corresponding text and layout information in a unified manner. Its autoregressive decoding mechanism is adept at capturing long-range dependencies, resulting in outputs that are coherent and contextually relevant to the original document. This streamlined, integrated approach simplifies the overall processing pipeline and reduces the error propagation commonly associated with multi-stage systems.

For our research, Donut's OCR-free methodology has important implications. By eliminating the need for separate text recognition steps, Donut enhances efficiency and robustness in document generation tasks. Its design demonstrates that end-to-end learning can effectively capture both the structural and textual aspects of documents, even under challenging conditions. Incorporating principles from Donut into our work could lead to more resilient document synthesis systems, especially when maintaining structural fidelity without relying on external OCR is critical.

3.3.3.DocSynthV2

DocSynthV2, developed by Biswas et al. [11] builds on the transformer architecture to address the dual challenges of generating coherent textual content and structured layouts within documents. By adopting an autoregressive modelling approach akin to the GPT series, it predicts elements sequentially, allowing for seamless integration of textual and layout features using decoder blocks present in the transformer. This architecture captures long-range dependencies and ensures contextual consistency, crucial for complex document generation tasks. Like T5's unified framework for NLP tasks, DocSynthV2 combines the generation of text and layout within a single modelling paradigm, simplifying task formulation while maintaining flexibility. Drawing from pre-training techniques in models like RoBERTa and GPT-3, it leverages large datasets and fine-tuning to adapt effectively across varied domains, ensuring both scalability and precision in document creation.

A significant limitation of DocSynthv2 is the unavailability of its model and the associated dataset, PubGenNet, to the public. PubGenNet was curated by extracting samples from the PubLayNet dataset, aligning layout and text data for document generation tasks. The lack of access to these resources poses challenges for researchers aiming to replicate or extend the work, as the proprietary nature of the model and dataset restricts broader experimentation and validation within the research community.

The lack of public access withstanding, Biswas' Et al.creates the most compelling research. The combination of autoregressive architecture and data curation/augmentation poises the work to be the greatest influence on our own research even without the ability to directly extend Biswas' work.

3.4. Literature Review Conclusion

3.4.1.Motivation

Although significant progress has been made in document automation and natural language generation—from rule-based systems and corpus-based methods to advanced deep neural and autoregressive models—several research gaps remain that motivate further investigation. For instance, while datasets such as PubLayNet [12] and DocBank [10] provide extensive annotated resources that have driven advances in document layout analysis, the effective transformation of unstructured inputs into fully structured outputs (e.g., generating XML representations) still presents challenges. Similarly, although recent approaches like DONUT [26] have streamlined document understanding by eliminating reliance on separate OCR modules, there remains an open question as to how intermediary representations (such as JSON or XML) might enhance the robustness and consistency of automated document generation.

In addition, while template-based systems have evolved to incorporate sophisticated linguistic knowledge, their integration with neural and autoregressive models remains a largely untapped potential for achieving both precision and scalability in real-world

applications. These unresolved issues underscore the need for a unified methodology that leverages the strengths of large language models, intermediary representations, and advanced training techniques. The following section presents our methodology, which is directly motivated by these research gaps and aims to develop a scalable, flexible, and high-quality document automation system.

4. Methodology

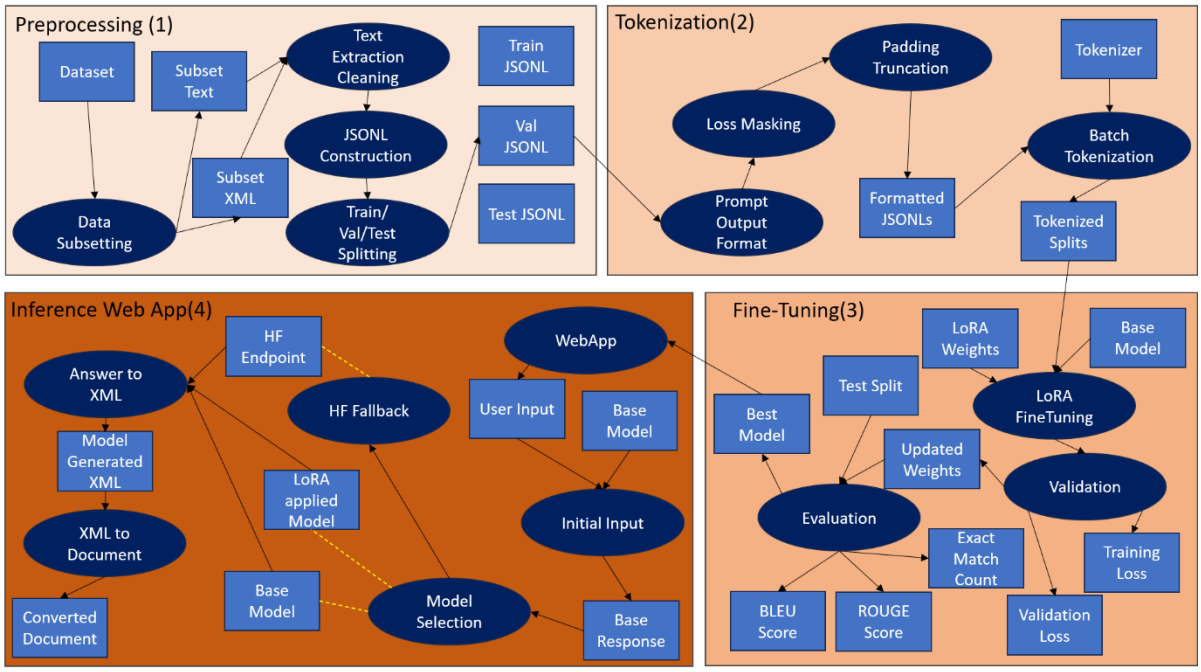


Figure 1 – Methodology Diagram

4.1. Dataset Preparation

4.1.1. Training Data

The core training data for this project is derived from the DocBank dataset [14], a large-scale corpus of 500,000 document pages from arXiv with fine-grained token-level layout annotations. DocBank provides each token's bounding box coordinates and its semantic category (e.g., title, paragraph, list item, figure caption, etc.), spanning 12 distinct element categories. This rich annotation makes it possible to reconstruct structured representations of documents. However, using the entire DocBank corpus is computationally impractical for the scope of the project. Therefore, curating a manageable subset suitable for language model training would be best practice.

Given constraints, a portion of the database would be required without loading all 500k pages into memory would be crucial to not only avoid exceeding the resources available but safeguard against information leakage through cross contamination of the training, testing and validation splits.

4.1.2. Text extraction and Cleaning

DocBank provides bounding boxes and labels for layout elements (e.g. paragraphs, sections, references) but not the full text content in an easily usable form. This would require matching and merging raw text for each selected annotation. Moreover, aligning text from the original documents (extracted from LaTeX or PDF sources) with annotated segments by identifying unique patterns (e.g. section titles, reference markers) via regular expressions. The next step of preprocessing is to clean the extracted text to remove artifacts from the PDF-to-text conversion this may include eliminating encoding remnants like "cid:..." or fixing ligatures so that the content closely matches document text.

4.1.3.JSONL Construction

Finally, for each selected document segment the framework looks to construct JSON Lines record containing the text and its corresponding layout annotation. The methodology aims to format each annotation in a PASCAL VOC-style XML snippet, including the filename, element category, and bounding box coordinates. Each JSONL line has the structure: {"text": "<segment_text>", "xml": "<annotation>...</annotation>"}. This paired the segment's plain text with the ground-truth XML for that segment. Storing the data in a JSONL file ensures straightforward to load into Python for preprocessing and model training. As is common practise, the methodology would create separate splits (train/validation/test) from this subset to enable model evaluation on unseen examples, ensuring no overlap of documents between splits.

The result of this preparation is a cleaned, structured dataset of document snippets. Each example in the dataset consists of a text excerpt (such as a paragraph, list, or reference block) and the target XML annotation marking its category and coordinates. This dataset format is conducive to training a language model to predict layout annotations from text alone.

4.2. Tokenization

4.2.1.Tokenizers

For tokenization, the proposed employ the pre-trained Sheared-LLaMA tokenizer associated with the base Sheared-LLaMA-1.3B model [28] without any additional tokenizer training. This tokenizer reuses LLaMA's original SentencePiece vocabulary and embedding mapping without additional training, ensuring seamless compatibility with the model's learned representations. The vocabulary comprises 32,000 subword units, a size determined during LLaMA2's initial pre-training to balance token coverage against model efficiency.

The tokenizer implements byte-pair encoding (BPE) through the SentencePiece library, merging the most frequent byte pairs iteratively to construct its 32k-token vocabulary. BPE begins with a character-level inventory and repeatedly joins the most common adjacent symbols into new tokens until the predefined vocabulary size is reached. This mechanism allows rare words to be decomposed into meaningful subword sequences rather than fallback to single characters or full words, improving both compression and generalization. SentencePiece treats all input as raw Unicode text without language-specific pre-tokenization, tokenizing XML markup purely as sequences of Unicode characters. Consequently, custom tags (e.g., <sep>) are broken into subword pieces corresponding to their constituent characters (for instance <, sep, and >). This approach preserves the structural boundaries of XML annotations while leveraging the existing vocabulary.

Byte-pair encoding has become the de facto standard for subword segmentation in large-scale pre-training, underpinning models such as GPT-3, RoBERTa, and LLaMA2. SentencePiece's end-to-end, language-independent design further streamlines preprocessing by obviating external tokenization tools while supporting multiple subword algorithms.

Leveraging this well-established tokenizer aligns the proposed methodology with best practices in contemporary LLM development.

4.2.2.Prompt-Output Format

During training, each sample's text and annotation should be combined into a single string with a clear delimiter. Prepending an instruction phrase and appended a special separator token before the output. In practice, this format can be seen in Figure 2.

```
"Generate XML annotation for the following excerpt:\n{excerpt}\n<sep>"
```

Figure 2 – Prepended Prompt

Followed by the ground-truth XML. Here, {excerpt} is the document text snippet, and <sep> is a literal separator tag to indicate the end of the prompt. This prompt template (instruction + excerpt + <sep>) was consistent across all training examples, so the model learned to expect the input text before <sep> and to produce XML after. The <sep> token (not originally in the vocabulary) is treated as a sequence of characters by the tokenizer (i.e., < + sep + >), but serves as an unambiguous boundary in the sequence.

4.2.3.Tokenization and Loss Masking

Hugging Face's datasets library is proposed as the solution to efficiently apply the tokenizer to the entire dataset. A mapping function (dataset.map) tokenized would combine each prompt-output string.

It is important to adjust the labels to ensure the model is only trained on the output portion. All token indices corresponding to the prompt (the instruction text, the excerpt, and the <sep> token) were set to -100 in the label tensor, which is the ignore index for loss computation in PyTorch. This masking means that when computing the loss, so that the model is *not* penalized for predicting the prompt tokens (which it was explicitly given); only the tokens of the XML output are considered. This technique follows the standard practice for training language models in an instruction-following setup, where only the completion (response) is the learning target.

4.2.4.Padding and Truncation

The framework employs a maximum sequence length that could accommodate the longest prompt+output examples in our data, which will be informed by an initial exploration of the document. Any sequences shorter than this length is to be padded (with <pad> tokens and corresponding attention mask set to ignore them), and any sequences longer than the limit should be truncated. Padding is done to the right (i.e. after the end of the sequence) to preserve the prompt-output structure. By padding/truncating during the mapping step, it will be ensured that batches could be formed without further dynamic padding overhead.

4.2.5.Batch Tokenization Efficiency

Using the Hugging Face tokenizer with the datasets library would allow the framework to tokenize the entire dataset swiftly in batches, leveraging vectorized operations. This approach aims to be much faster and less error-prone than manual Python looping. Equally ensuring consistency in how text and XML will be broken into tokens. Special characters in the XML (angle brackets, slashes, etc.) are to be handled by the tokenizer's byte-pair encoding, and no custom tokens aside from regular vocabulary were required, the sequence <sep> will be tokenized into normal subword tokens. The final output of this stage is to be a processed Dataset object ready for model training, with fields for input_ids, attention_mask, and labels. By the end of tokenization, each training example will be represented as a sequence of token IDs that encapsulate both the document excerpt and the expected XML annotation. The use of a pre-existing tokenizer ensures compatibility with the model's embeddings, and the loss masking attempts to ensure that the model would learn to generate the XML given the text, rather than simply echo the prompt.

4.3. Fine-Tuning

4.3.1.LoRA Fine-Tuning with SHEARED LLaMA

For the language model, the framework seeks to select the Princeton Sheared LLaMA as the base. Specifically, the Sheared-LLaMA 1.3B model (released by Mengzhou Xia and colleagues in 2023 [28]), a variant distilled from LLaMA2-7B via structured pruning. This model is attractive for our task because it retains much of LLaMA2's capability but at only 1.3 billion parameters, making fine-tuning feasible on a single GPU. Sheared-LLaMA was derived by pruning a larger model and then further pre-training it on 50 billion tokens [28], yielding a smaller model that "outperforms existing large language models" of similar scale.

Importantly, it uses the same vocabulary and architecture as LLaMA2, which means the approach can leverage its weights directly and integrate the original tokenizer with no adjustments. The choice of a 1.3B model is also dictated by compute considerations a model larger would be difficult to finetune without multi-GPU or out-of-core techniques. The 1.3B Sheared LLaMA, however, fits in memory even with additional fine-tuning overhead, especially when using 16-bit or 8-bit precision.

The project aims to fine-tune the Sheared LLaMA on the subsetted dataset using Low-Rank Adaptation (LoRA) [29], a parameter-efficient fine-tuning technique. LoRA freezes the original model weights and injects small trainable weight matrices (of low rank) into each transformer layer. Essentially, instead of updating the full weight of a neural network layer, which would be huge for a model of this size. LoRA introduces two much smaller matrices is effectively augmented by during training. This drastically reduces the number of trainable parameters – often by tens of thousands of times – while still allowing the model to adapt to new tasks. It is optimum to set the LoRA rank for the attention projector layers of the model. This means only a few million additional parameters (at most) are introduced, which is a tiny fraction (<1%) of the 1.3B base model when targeting the query and value projection matrices of each self-attention block, as suggested by Hu *et al.* (2021) [29], since these have been found effective for language model adaptation. All other weights (most of the model) remained frozen, saving memory and preserving the original model's knowledge.

To implement LoRA, Hugging Face's PEFT (Parameter-Efficient Fine-Tuning) library [30] PEFT provides a convenient API to wrap a pre-trained model with LoRA modules. First loading the Sheared LLaMA weights via the Hugging Face Transformers library (Wolf *et al.*, 2020) [31] and then used peft.LoraConfig to specify the LoRA setup such as task type and target modules. The model is then converted to a PeftModel which internally handles the forward pass combining base weights and LoRA weights. One advantage of this approach is that the model remains lightweight in memory: the base 1.3B parameters can be kept in 16-bit precision (or even 8-bit using the bitsandbytes library) and the LoRA adds negligible overhead, well within Colab's GPU limits. Enabling 8-bit weight quantization for the base model using bitsandbytes, should further reduce memory usage such that the total model (base + LoRA) uses even less VRAM. Which should result in ample room for training a reasonable batch size.

4.3.2.Training configuration

The approach looks to fine-tune the model in Google Colab for several epochs over the training subset. Using an AdamW optimizer with a learning rate for the LoRA parameters. – this rate is relatively high compared to full-model fine-tuning, but LoRA layers are small and benefit from a slightly higher learning rate as noted by Hu et al. [29]. It is important to consider the learning rate, linear decay and batch size. The Learning rate should not be static

as to avoid missing subsections of the solution search space. The learning decay can help achieve this, but it is always important to experiment with different combinations such as a warmup. Equally a batch size should be achieved by balancing gradient accumulation, sequences per step and number. These can promote both efficient and effective training while adhering to resource or environment limits.

During training itself, early stopping can be employed when measured against the stagnation of validation loss to prevent overfitting. Throughout training, it is key to monitor the validation loss. Further checks could include periodically generating a few sample outputs from the model to qualitatively check progress, but this would need to be established and would interrupt training time itself.

Ensuring the model learns *only* to generate the XML given the text is critical, the approach aims to avoid the model trying to regenerate the input text itself which will be provided with the instruction. To enforce this, applying an appropriate causal mask in the loss function as well as prepending a brief instruction. The final fine-tuned model should be the original Sheared LLaMA plus a set of LoRA matrices. These LoRA weights can then be exported such that anyone with the same base model can replicate the approach by applying the adapter which should be much smaller to distribute than the full model checkpoint.

4.4. Flask Web App

4.4.1.Web App

In parallel to the model being fine-tuned, the approach will build a lightweight Flask web application to deploy it for interactive use.

4.4.2.Local Inference First

The deployment is designed to be robust by using local inference when possible and falling back to a remote service if needed, as well as to ensure the output is formatted correctly.

On server startup, the Flask app will load the fine-tuned model along with the tokenizer and LoRA weights into memory. If a GPU is available on the host machine, the model is moved to the GPU for faster inference. When a user submits an excerpt via the web interface, the app will run the model locally to generate the XML. Inference will be done using the model's .generate() function from Hugging Face Transformers. The app will construct the input in the same format used during training. The model can then autoregressively produce an output sequence of tokens to decode into text. This local inference aims to provide low-latency responses and does not require internet access.

4.4.3.Fallback to Hugging Face API

In cases where local inference is not possible – for example, if the app is deployed on a machine without a suitable GPU, or if the model fails to load due to memory constraints. The approach will implement a fallback to Hugging Face's hosted inference API. The fine-tuned model will be uploaded to the Hugging Face Hub as a private model to be call the Hugging Face Inference Endpoint to run the model on their servers. In the Flask code, this should be handled by catching exceptions from the local generation. If an error occurs or a configuration flag is set to force remote, the app sends an API request to Hugging Face with the user's input and our specified generation parameters. The API returns the model-predicted text as a JSON response, which the Flask app then relays back to the user. The app will include this fallback to ensure the demo is functional even in resource-constrained deployments. The web interface will clearly indicate the source of the response, labelling it as "(generated locally)" or showing the raw HF response for transparency.

4.4.4.Prompt Template and Generation Setting

The prompt template used in the Flask app is identical to what was used during training. For a given user input excerpt, the app formats the model input as is seen in Figure 2.
This consistency ensures the model receives input in the form it was fine-tuned on, which is crucial for obtaining correct outputs. In generating the response, configuration could set decoding parameters to favour deterministic, structured outputs. Equally employing maximum tokens for the output, which would plenty for a single XML annotation. Disabling temperature could supress excessive randomness while enable nucleus sampling to allow some variability if needed.
Since the task is quite deterministic (there's essentially one correct annotation for a given text), the model's outputs should be stable. The generation parameters are chosen to prevent the model from trailing off into irrelevant text beyond the XML. Relying on the model's familiarity with the XML format tokens to know when to terminate.

4.4.5.Input to Response to XML

Rather than directly transforming user-supplied text excerpts into layout annotations, the proposed methodology seeks to incorporate an intermediate step that reframes user input as a natural language request. This input may take the form of a question, instruction, or prompt, designed to elicit a structured textual response from a general-purpose language model. The intent of this design would be to decouple semantic content generation from the annotation process, enabling the system to first generate a coherent and contextually appropriate response based on the user's intent before applying the fine-tuned model.

To facilitate this pipeline, the user's input would first be sanitized and normalized to remove extraneous characters, ambiguous constructions, or unsupported input types. This cleaned version of the query would then be passed to a language model—either deployed locally or accessed via a hosted inference endpoint—capable of producing fluent, well-formed natural language responses. The resulting output would then serve as the effective "text excerpt" to be annotated, and would subsequently be passed through the fine-tuned model to generate its corresponding XML layout representation.

This two-stage process is intended to introduce a modular abstraction layer between intent interpretation and structured annotation. By chaining a semantic response model with a layout annotation model, the framework aims to support a more dynamic and intelligent document generation pipeline—where layout-aware annotations are not only applied to existing content, but generated in response to interactive user queries. This extension is expected to broaden the system's applicability to tasks involving both content creation and structural formatting.

4.4.6.XML Conversion to Document

Following generation, the XML output is to be processed to ensure it conforms to a valid and well-structured format. This may involve minor adjustments such as pruning invalid or incomplete tags, correcting mismatched elements, and cleaning up any malformed components that arise during decoding. These steps aim to ensure the output adheres to standard XML syntax, which is essential for subsequent conversion tasks.

Once validated, the XML is intended to be transformed into a structured document format, such as a Word (.docx) or PDF file. This would require parsing the XML and mapping each annotation tag to its corresponding layout element within the document. For instance, annotated categories like paragraphs, headings, and references would be rendered using predefined style templates.

The resulting document would then be made available for user download via the web interface. This step is intended to provide an end-to-end workflow—from raw text input to a complete, stylized document—enabling users to obtain an editable or shareable representation of the model's layout-aware annotations.

4.5. Evaluation Metrics

4.5.1.XML Well-Formedness

To verify whether the model's output is well-formed XML, which in practise is means that all tags opened are properly closed and nested, and the output can be parsed by an XML parser without errors.

This could be implemented by leveraging Python's built-in XML parsing library (xml.etree.ElementTree). For each generated output, the approach can attempt to parse it; if the parser throws an error (due to mismatched or malformed tags), this is then recorded that output as *not* well-formed.

By computing the percentage of outputs in the test set that are well-formed. This becomes crucial metric because an output that is not even syntactically valid XML would be unusable in a downstream document processing pipeline. Ensuring well-formedness is our primary sanity check as it reflects the model's adherence to the structural *format* required.

4.5.2.BLEU Score

We use the BLEU metric [32], which is noted through figure 3 and 4, to measure how closely the generated XML matches the reference XML for each example. BLEU (Bilingual Evaluation Understudy) is a precision-oriented metric originally developed for machine translation. It computes the overlap of n-grams (up to 4-grams typically) between the candidate text and one or more reference texts, combined via a weighted geometric mean, with a penalty for too-short outputs. This penalty is seen in figure 3.

$$\mathrm{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{1-\frac{r}{c}} & \text{if } c \leq r \end{cases}$$

Figure 3 – Brevity Penalty Equation

$$\mathrm{BLEU} = \mathrm{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

Figure 4 – BLEU Score Equation

In the framework's context, the reference XML is treated as the reference text and the model output as the candidate. A high BLEU score means the model output has a lot of exact n-gram matches with the reference, implying it captured not only the correct words but also the correct arrangement of tags relative to words. BLEU is effective as it heavily penalizes deviations in text/content; for instance, if the model misses a section of text or a tag, the n-gram overlap will decrease. However, BLEU alone might be harsh for this task – even a structurally correct output could get a lower BLEU if, say, a single word is different (perhaps the model fixed a typo or spacing). Therefore, the approach uses it in conjunction with ROUGE to get a more balanced view.

4.5.3.ROUGE Scores

$$\text{ROUGE-N} = \frac{\sum_{S \in \text{Ref}} \sum_{gram_n \in S} \text{Count}_{\text{match}}(gram_n)}{\sum_{S \in \text{Ref}} \sum_{gram_n \in S} \text{Count}(gram_n)}$$

Figure 6 – ROUGE-N Equation

$$\text{ROUGE-L} = \frac{(1+\beta^2) \cdot \text{R}_{\text{LCS}} \cdot \text{P}_{\text{LCS}}}{\text{R}_{\text{LCS}} + \beta^2 \cdot \text{P}_{\text{LCS}}} \text{R}_{\text{LCS}} = \frac{\text{LCS}(X,Y)}{\text{length}(Y)}, \quad \text{P}_{\text{LCS}} = \frac{\text{LCS}(X,Y)}{\text{length}(X)}$$

Figure 5 – ROUGE-L Equation

The methodology also evaluates using ROUGE (Recall-Oriented Understudy for Gisting Evaluation), particularly ROUGE-L as seen in Figure 5 and ROUGE-N. in figure 6. ROUGE was originally introduced for summarization tasks (Lin, 2004) as a recall-focused metric, measuring how much of the reference text appears in the candidate [33]. The framework would compute ROUGE-N for N=1,2 (i.e., overlap of unigrams and bigrams) and ROUGE-L (Longest Common Subsequence). ROUGE will reward the model for capturing as many of the reference tokens as possible, regardless of order, as the model might sometimes reorder minor things or produce the same content with slight differences in formatting. For example, if the model output has all the same words as the reference but maybe slightly different line breaks or a tag order swap that doesn't change content, ROUGE could still be high even if BLEU is somewhat lower. A high ROUGE-L in particular means the model's output shares a long common subsequence with the reference, indicating it preserved the overall content and sequence of information well. This is very pertinent for structured documents, where all parts of the text should be present in the output.

Together, BLEU and ROUGE provide a complementary picture: BLEU emphasizes exact precision while ROUGE emphasizes recall. In the context of structured XML, the approach assumes both to be high if the model performs well since the ideal is the model outputs exactly the reference.

In cases where they diverge, it might drop or alter a minor element, which is reflected in slightly lower ROUGE.

These metrics treat the XML tags just like words, so a missing or incorrect tag directly impacts the scores. This is appropriate as an XML tag is a critical token in the sequence. For instance, if the model outputs <section>Introduction</section> but the reference was <title>Introduction</title>, many n-grams will not match, reducing both BLEU and ROUGE, which correctly signals a significant error in the structured output.

5. Results

5.1. Data Preparation Results

5.1.1.Original JSON Files


{"info": {"year": 2020, "version": "1.0", "description": "The MS COCO format version of DocBank.", "contributor": "Minghao Li, Yiheng Xu, Lei Cui, Shaohan Huang,
Furu Wei, Zhoujun Li, Ming Zhou", "url": "https://github.com/doc-analysis/DocBank"},
"images": [{"id": 1, "width": 773, "height": 1000, "file_name": "1.tar_1401.0001.gz_infoingames_without_metric_arxiv_47_ori.jpg", "license": 1}, {"id": 2, "width": 773,
"height": 1000, "file_name": "1.tar_1401.0001.gz_infoingames_without_metric_arxiv_8_ori.jpg", "license": 1}, {"id": 3, "width": 707, "height": 1000, "file_name":
"1.tar_1401.0019.gz_sup-set-finalversion_6_ori.jpg", "license": 1}, {"id": 4, "width": 773, "height": 1000, "file_name":
"1.tar_1401.0023.gz_abramson2013-emulateapj-12-30_11_ori.jpg", "license": 1}, {"id": 5, "width": 707, "height": 1000, "file_name": "1.tar_1401.0052.gz_bigmainarxiv_9_ori.jpg",
"license": 1}, {"id": 6, "width": 773, "height": 1000, "file_name": "1.tar_1401.0088.gz_Tau_Complete_Factorization_14_ori.jpg", "license": 1}, {"id": 7, "width": 773,
"height": 1000, "file_name": "1.tar_1401.0091.gz_Mooney_-_Tau_Regular_Factorization_RMJM_8_ori.jpg", "license": 1}, {"id": 8, "width": 707, "height": 1000, "file_name":
"1.tar_1401.0105.gz_cohesive_13_ori.jpg", "license": 1}, {"id": 9, "width": 773, "height": 1000, "file_name": "1.tar_1401.0127.gz_optimal_stabilization_12_ori.jpg",
"license": 1}, {"id": 10, "width": 773, "height": 1000, "file_name": "1.tar_1401.0157.gz_aperiodic_revised_11_ori.jpg", "license": 1}, {"id": 11, "width": 773,
"height": 1000, "file_name": "1.tar_1401.0174.gz_s_voronoi_4_ori.jpg", "license": 1}, {"id": 12, "width": 773, "height": 1000, "file_name": "1.tar_1401.0189.gz_Depth3_7_ori.jpg",
"license": 1}, {"id": 13, "width": 773, "height": 1000, "file_name": "1.tar_1401.0202.gz_lamperskioptimal2013_rev05_11_ori.jpg", "license": 1}, {"id": 14, "width": 707,
"height": 1000, "file_name": "1.tar_1401.0209.gz_fullpaper_13_ori.jpg", "license": 1}, {"id": 15, "width": 686, "height": 1000,
"file_name": "1.tar_1401.0217.gz_CLE-Extremes_41_ori.jpg", "license": 1}, {"id": 16, "width": 773, "height": 1000, "file_name":
"1.tar_1401.0220.gz_Trees_in_Sparse_Graphs_10_ori.jpg", "license": 1}, {"id": 17, "width": 707, "height": 1000, "file_name": "1.tar_1401.0221.gz_revised_paper_19feb_3_ori.jpg",
"license": 1}, {"id": 18, "width": 773, "height": 1000, "file_name": "1.tar_1401.0224.gz_arxiv_12_ori.jpg", "license": 1}, {"id": 19, "width": 773, "height": 1000, "file_name":
"1.tar_1401.0224.gz_arxiv_25_ori.jpg", "license": 1}, {"id": 20, "width": 773, "height": 1000, "file_name": "1.tar_1401.0230.gz_LagrandisRevision_22_ori.jpg", "license": 1},
{"id": 21, "width": 707, "height": 1000, "file_name": "1.tar_1401.0231.gz_fractal_distributions-GMT_FINAL_9_ori.jpg", "license": 1}, {"id": 22, "width": 773, "height": 1000,
"file_name": "1.tar_1401.0232.gz_FinitenessofAttractorsArXivV2_31_ori.jpg", "license": 1}, {"id": 23, "width": 773, "height": 1000, "file_name": "1.tar_1401.0235.gz_main4_4_ori.jpg",
"license": 1}, {"id": 24, "width": 707, "height": 1000, "file_name": "1.tar_1401.0252.gz_Manuscript_revised_8_ori.jpg", "license": 1}, {"id": 25, "width": 773, "height": 1000,
"file_name": "1.tar_1401.0271.gz_NLS_with_network_patern_24112012_46_ori.jpg", "license": 1}, {"id": 26, "width": 773, "height": 1000, "file_name":
"1.tar_1401.0271.gz_NLS_with_network_patern_24112012_52_ori.jpg", "license": 1}, {"id": 27, "width": 773, "height": 1000, "file_name": "1.tar_1401.0277.gz_trans_17_ori.jpg",
"license": 1}, {"id": 28, "width": 773, "height": 1000, "file_name": "1.tar_1401.0288.gz_bh_bdry09_4_ori.jpg", "license": 1}, {"id": 29, "width": 707, "height": 1000,
"file_name": "1.tar_1401.0339.gz_paper_13_ori.jpg", "license": 1}, {"id": 30, "width": 773, "height": 1000, "file_name": "1.tar_1401.0364.gz_Journal_Paper_10_ori.jpg", "license": 1},
{"id": 31, "width": 773, "height": 1000, "file_name": "1.tar_1401.0379.gz_tromp_numbers_4_ori.jpg", "license": 1}, {"id": 32, "width": 773, "height": 1000, "file_name":
"1.tar_1401.0397.gz_eqMarkov_4_ori.jpg", "license": 1}, {"id": 33, "width": 773, "height": 1000, "file_name": "1.tar_1401.0409.gz_LongRangePerc_arXiv_3_ori.jpg",
"license": 1}, {"id": 34, "width": 707, "height": 1000, "file_name": "1.tar_1401.0429.gz_Critical_BRW_arxiv3_7_ori.jpg", "license": 1}, {"id": 35, "width": 707, "height": 1000, "file_

Figure 7 – Original JSON file of DocBank Annotations

Each JSON file in the dataset represented a single annotated document page and served as a foundational element for the training pipeline developed in this research. These files were originally curated as part of the original DocBank Research [26], and each one contained a structured XML-style annotation that described the spatial and semantic properties of individual text elements found within the corresponding document image.

Specifically, the JSON entries encapsulated metadata such as the coordinates and dimensions of each annotated region as seen in Figure 7. The semantic label assigned to that region (e.g., section, title, figure), and the filename linking the annotation to its source text file. This tight coupling between visual layout data and textual content made the dataset especially well-suited for training a model to learn structured document representations.

A typical annotation followed a standardized format, enclosed within a root <annotation> tag. This structure included:

- A <filename> tag identifying the original document image.
- A <size> block detailing the width, height, and depth (color channels) of the page.
- One or more <object> blocks, each describing an individual annotated element. Within these blocks, a <name> tag indicated the category of the item (e.g., “section”), while a nested <bndbox> tag provided the bounding box coordinates (xmin, ymin, xmax, ymax) specifying the item’s position on the page.

  The Structure can be further examined in Figure 8.

Figure 8 – DocBank XML Annotation

This illustrates the level of detail provided in each annotation, allowing the model to associate

```
<annotation>
  <filename>96.tar_1506.06731.gz_corr39_10_ori.jpg</filename>
  <size>
    <width>773</width>
    <height>1000</height>
    <depth>3</depth>
  </size>
    <object>
        <name>section</name>
            <bndbox>
                <xmin>168</xmin>
                <ymin>761</ymin>
                <xmax>190</xmax>
                <ymax>779</ymax>
        </bndbox>
    </object>
</annotation>
```

raw text with its precise spatial and semantic context. These structured XML annotations formed the ground truth targets during training, enabling the model to learn how to generate similar markup directly from plain text inputs.

5.1.2.Original Text Files

99.tar_1705.03840.gz_n-hereditary_rings_and_torsion_theories_10_2	03/07/2020 09:31	Text Document	28 KB
99.tar_1705.03841.gz_DomainWall_0	03/07/2020 09:31	Text Document	67 KB
99.tar_1705.03841.gz_DomainWall_8	03/07/2020 09:31	Text Document	131 KB
99.tar_1705.03841.gz_DomainWall_9	03/07/2020 09:31	Text Document	59 KB
99.tar_1705.03841.gz_DomainWall_11	03/07/2020 09:31	Text Document	6 KB
99.tar_1705.03851.gz_rot_10	03/07/2020 09:31	Text Document	24 KB
99.tar_1705.03852.gz_adaptive_13	03/07/2020 09:31	Text Document	25 KB

Figure 9 – Contents of DocBank_txt folder

The associated text content was sourced from the DocBank_500k_txt folder as seen in Figure 9, which houses over 16GB of plain text files derived from LaTeX-based arXiv documents. Due to the volume and file count, initial extraction processes were computationally expensive and suffered from performance bottlenecks. To address this, a multithreaded extraction strategy was implemented.

In reference to Figure 10, Each text file contains multiple lines of document excerpts. The file

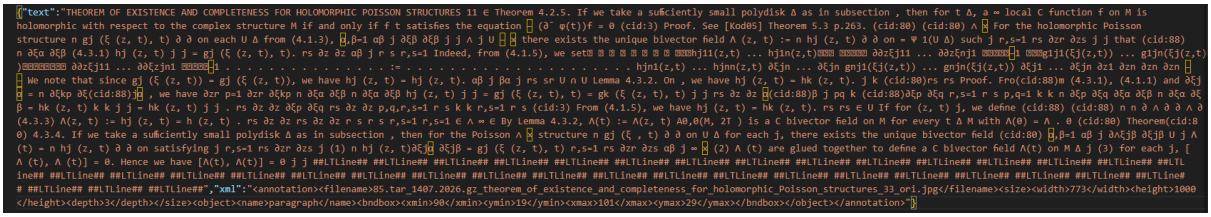
Value	237	211	301	237	0	0	0
of	308	211	332	237	0	0	0
information	339	211	472	237	0	0	0
in	479	211	501	237	0	0	0
noncooperative	508	211	680	237	0	0	0
games	687	211	759	237	0	0	0
*	350	249	356	267	0	0	0
†	512	249	519	267	0	0	0
‡	675	249	681	267	0	0	0
Nils	212	256	245	274	0	0	0

Figure 10 – Contents of Example Text document from folder

names were programmatically matched against those found in the JSON annotations to ensure

correspondence between text and layout. Extracting these matched excerpts formed the basis for constructing training samples used in model fine-tuning.

5.1.3.Matched Text XML Pairs


{"text":"THEOREM OF EXISTENCE AND COMPLETENESS FOR HOLOMORPHIC POISSON STRUCTURES 11 ∈ Theorem 4.2.5. If we take a sufficiently small polydisk Δ as in subsection , then for t Δ, a ∞ local C function f on M is
holomorphic with respect to the complex structure M if and only if f t satisfies the equation □ (∂ˉ φ(t))f = 0 (cid:3) Proof. See [Kod05] Theorem 5.3 p.263. (cid:80) (cid:80) Λ □ for the holomorphic Poisson
structure n gj (ξ (z, t), t) ∂ ∂ on each U Δ from (4.1.3), □,β=1 αβ j ∂ξβ ∂ξβ j j Λ j U □ □ there exists the unique bivector field Λ (z, t) :∼ n hj (z, t) ∂ ∂ on ∼ Ψ 1(U Δ) such j r,s=1 rs ∂zr ∂zs j j that (cid:88)
n ∂ξα ∂ξβ (4.3.1) hj (z, t) j j ∼ gj (ξ (z, t), t). rs ∂z ∂z αβ j r s r,s=1 Indeed, from (4.1.5), we set□ □ □ □ □ □ □ □□□□hj11(z,t) ... hjin(z,t)□□□ □□□□□□ ∂∂zξj11 ... ∂∂zξnj1 □□□□□□□1 □□□gij1(ξj(z,t)) ... gijn(ξj(z,t)
)□□□□□□□□ ∂∂zξj11 ... ∂∂ξzjn1 □□□□□□1 . . . . . :∼ . . . . . hjn1(z,t) ... hjnn(z,t) ∂ξjn ... ∂ξjn gnj1(ξj(z,t)) ... gnjn(ξj(z,t)) ∂ξj1 ... ∂ξjn ∂z1 ∂zn ∂zn ∂zn □
□ We note that since gj (ξ (z, t)) ∼ gj (ξ (z, t)), we have hj (z, t) ∼ hj (z, t). αβ j βα j rs sr U n U Lemma 4.3.2. On , we have hj (z, t) ∼ hk (z, t). j k (cid:80)rs rs Proof. Fro(cid:80)m (4.3.1), (4.1.1) and ∂ξj
□ ∼ n ∂ξkp ∂ξ(cid:88)□□ , we have ∂zr p=1 ∂zr ∂ξkp n ∂ξα ∂ξβ n ∂ξα ∂ξβ hj (z, t) j j ∼ gj (ξ (z, t), t) ∼ gk (ξ (z, t), t) j j rs ∂z ∂z □(cid:88)∂ξp ∂ξq r,s=1 r s p,q=1 k k n ∂ξp ∂ξq ∂ξα ∂ξβ n ∂ξα ∂ξ
β ∼ hk (z, t) k k j j ∼ hk (z, t) j j . rs ∂z ∂z ∂ξp ∂ξq rs ∂z ∂z p,q,r,s=1 r s k k r,s=1 r s (cid:3) From (4.1.5), we have hj (z, t) ∼ hk (z, t). rs rs ∈ U If for (z, t) j, we define (cid:88) (cid:80) n n ∂ Λ ∂ ∂ ∧ ∂
(4.3.3) Λ(z, t) :∼ hj (z, t) ∼ h (z, t) . rs ∂z ∂z rs ∂z ∂z r s r s r,s=1 r,s=1 ∈ Λ ∼ ∈ By Lemma 4.3.2, Λ(t) :∼ Λ(z, t) A0,0(M, 2T) is a C bivector field on M for every t Δ M with Λ(0) ∼ Λ . 0 (cid:80) Theorem(cid:8
0) 4.3.4. If we take a sufficiently small polydisk Δ as in subsection , then for the Poisson Λ □ structure n gj (ξ , t) ∂ ∂ on U Δ for each j, there exists the unique bivector field (cid:80) □,β=1 αβ j ∂Λξjβ ∂ξjβ U j Λ
(t) ∼ n hj (z, t) ∂ ∂ on satisfying j r,s=1 rs ∂zr ∂zs j (1) n hj (z, t)∂ξj□ ∂ξjβ ∼ gj (ξ (z, t), t) r,s=1 rs ∂zr ∂zs αβ j ∼ □ (2) Λ (t) are glued together to define a C bivector field Λ(t) on M Δ j (3) for each j, [
Λ (t), Λ (t)] = 0. Hence we have [Λ(t), Λ(t)] = 0 j j ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTL
ine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTLine## ##LTL

Following preprocessing, the cleaned text-XML pairs were converted into structured, instruction-based prompts. This can be seen in Figure 13, which was designed to guide the model during fine-tuning. Each example was embedded within a consistent template that framed the task clearly.

This prompt format not only helped standardize the input across the training set but also reinforced the instructional nature of the task, signalling to the model that it should produce a structured XML output based on the given document excerpt.

To prepare the data for model ingestion, all prompts were tokenized using the Sheared-LLaMA tokenizer, which is derived from LLaMA's SentencePiece vocabulary. This ensured full compatibility with the underlying architecture and preserved subword integrity across both natural language and XML tokens.

As part of the tokenization process, label masking was applied to prevent the model from being trained on the prompt itself. Tokens corresponding to the instruction text, the document excerpt, and the <sep> delimiter were assigned a loss mask value of -100, which can be seen in the label slice of figure 13. Excluding them from contributing to the loss function during training. In this way, the model was explicitly trained to focus solely on the generation of the XML output.

To support batch training, all sequences were padded and truncated to a fixed maximum length, ensuring consistency across input sizes. This preprocessing step was efficiently handled by the Hugging Face datasets library, which allowed for vectorized, batched tokenization—significantly reducing overhead and enabling seamless integration with downstream training pipelines.

5.1.6.Training Configurations

Experiment	Dataset	Dataset Size	Initial Learning rate	Number of Epochs	Quantization	Batch Size	Gradient Accumulation	LoRA Rank	LoRA Alpha	Target Modules	Tokenizer Resize
Finetune_50k	Text/XML Pairs	50,000	2.00E-05	5	8-bit	8	4	8	16	Query/Value Projections	Yes
InstructTune_50k	Instruction/Text/XML Triplets	50,000	2.00E-05	5	8-bit	8	4	8	16	Query/Value Projections	No
InstructTune_100k	Instruction/Text/XML Triplets	100,000	2.00E-05	5	8-bit	8	4	8	16	Query/Value Projections	No

Figure 14 – Training Configuration Table

To evaluate the ability of large language models to generate structured XML annotations from raw text, a series of controlled fine-tuning experiments were conducted using three distinct configurations. These experiments were designed to assess the impact of instruction tuning, dataset scale, and architecture parameters on model performance and output quality. All training was performed using the Sheared-LLaMA 1.3B model as the base, with LoRA (Low-Rank Adaptation) employed to enable parameter-efficient fine-tuning.

The three configurations as shown in Figure 14 were as follows:

- Finetune_50k, which used 50,000 paired examples of document text and XML annotations without any instructional prompting.
- InstructTune_50k, which used the same number of examples but formatted each input as a triplet: an instruction prompt, followed by the document excerpt and its associated XML output.
- InstructTune_100k, which expanded the instruction tuning strategy to a larger dataset of 100,000 triplets.

All models were trained for 5 epochs using a consistent initial learning rate of $2 \times 10^{\{-5\}}$

and 8-bit quantization to reduce memory overhead during training. A batch size of 8 was used across the board due to GPU memory limitations, with gradient accumulation steps set to 4, effectively increasing the gradient update window to simulate a larger batch size without breaching resource constraints.

For LoRA configuration, all models applied rank 8 low-rank adaptation with a LoRA alpha of 16. The fine-tuning specifically targeted the query and value projection layers of each transformer block—layers known to have high impact on attention dynamics while keeping the number of trainable parameters minimal. This setup allowed for high adaptability with a small footprint, ensuring that fine-tuning could be carried out efficiently even within the constraints of a single GPU environment such as Google Colab.

A notable distinction among the configurations lies in tokenizer resizing. In the Finetune_50k setup, the tokenizer was explicitly resized to accommodate custom tokens (such as <sep>), which were introduced to mark boundaries between prompts and outputs. In contrast, the instruction-tuned models used the default tokenizer configuration without resizing, relying instead on the model to generalize subword representations for special tokens using the existing vocabulary. This difference may have impacted how reliably each model recognized structural delimiters in the input-output sequences.

The dataset formatting also played a key role. The Finetune_50k model was trained on simple input-output pairs with no instructional prefix, relying solely on the implicit structure of the data. In contrast, both instruction-tuned models used a consistent prompt template to frame each input as seen in Figure 15.

```
Generate XML annotation for the following excerpt:
{document excerpt}
<sep>
```

Figure 15 – Prompt Template

This approach was intended to steer the model more explicitly toward the desired output format by incorporating instruction-following behavior into the training process. However, as later results suggest, the additional instruction may have introduced complexity that hindered the model's precision in XML generation, particularly when the dataset was small.

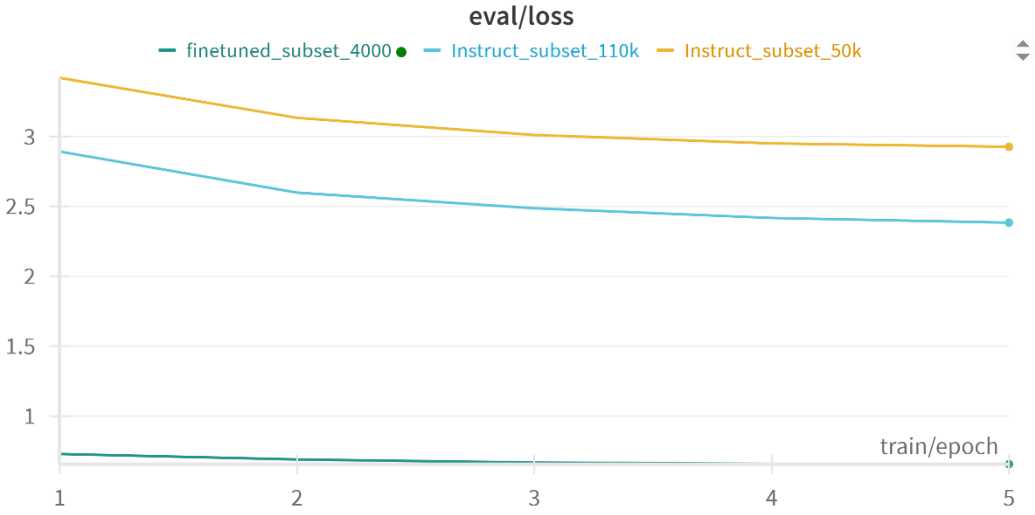
All three configurations employed the same tokenization and masking strategy during training: each sequence was tokenized into a unified input string, and only the output tokens (i.e., those after the <sep> delimiter) were included in the loss calculation, ensuring that the model was trained exclusively to generate the XML and not to reconstruct the prompt or source text.

This suite of training runs provided a robust foundation for evaluating not just output quality in terms of BLEU and ROUGE scores, but also structural accuracy as assessed by XML well-formedness—an essential requirement for practical deployment in document automation tasks. The differences in model setup, including dataset size, prompt structure, tokenizer behaviour, and LoRA configuration, all contributed to nuanced variations in performance, which are examined in detail in the results section.

5.2. Validation Scores

An analysis of the validation loss across different training configurations revealed several consistent patterns. Among all models, the 50k fine-tuned variant without instruction

Figure 16 – Comparison of Model Loss across Epoch



prompting demonstrated the fastest convergence and achieved the lowest overall validation loss as shown by Figure 16 above. This suggests that a direct, unprompted pairing of text with its corresponding XML annotation was the most efficient setup for the model to learn from.

In contrast, the 100k instruction-tuned model did show some improvement over its smaller 50k counterpart, likely due to the increased training data. However, despite the larger dataset, it still failed to outperform the baseline fine-tuned model in terms of validation loss. This finding indicates that the added complexity introduced by instructional prompts did not translate into better generalization or learning stability.

One possible explanation lies in the increased sequence length and structural redundancy introduced by the instructional formatting. While instruction tuning is widely regarded as a beneficial technique for general-purpose language models, in this highly constrained and structurally deterministic task—generating well-formed XML from specific document excerpts—it appeared to offer little tangible advantage. In fact, the instructional overhead may have introduced noise or confusion, particularly when the model struggled to generalize the prompt structure consistently across examples.

Taken together, these results suggest that while instruction tuning remains a powerful paradigm for open-ended generative tasks, it may not yield substantial benefits for narrowly defined objectives like structured document generation, where the target output is relatively rigid and formulaic.

5.3. GridSearch Optimisation

Parameter	Configurations		
Number of Beams	1	4	8
No Repeat ngram size	2	4	6
repitition penalty	0.6	1	1.5
max new tokens	50	80	100

Figure 17 – Grid Search Optimisation Search Space

A comprehensive GridSearch was performed to fine-tune key generation hyperparameters, with a particular focus on beam size and maximum token limits, both of which have a

significant impact on output quality and structural fidelity. This evaluation was essential for optimizing the model's ability to generate precise, well-structured XML annotations from text excerpts.

With regards to Figure 17, the study found that a beam size of 4 consistently produced the most balanced results. This setting effectively navigated the trade-off between precision and diversity. Increasing the beam size to 8 often led to outputs that were unnecessarily verbose or prone to repeating empty tags. Conversely, lower beam sizes constrained the model's expressive capability, often producing overly conservative or repetitive outputs with limited variation.

The maximum number of new tokens generated also proved to be a sensitive parameter. At a ceiling of 100 tokens, the model frequently produced trailing sequences that were either unrelated to the original content or redundantly extended the annotation. This behaviour highlights a deeper issue: the model did not consistently learn to terminate its output at appropriate structural boundaries, such as </annotation> or </object>. As a result, the importance of this parameter is currently more reflective of a workaround than an optimal tuning opportunity.

Given the overall limitations in generation quality, particularly with regard to structural completeness and XML well-formedness, it remains uncertain whether max token tuning alone can yield substantial improvements. Future models that better internalize explicit end-of-sequence tags may render max token constraints less critical, allowing the model to self-regulate output length more reliably.

5.4. Evaluation Metrics

5.4.1.XML Wellformedness Scores

All model configurations—regardless of dataset size or prompt structure—failed to achieve a single instance of well-formed XML output. This striking result underscores a critical limitation in the current approach. The XML well-formedness metric, by design, is binary and unforgiving: even a single misplaced or unclosed tag, an extra character, or a malformed structure result in an invalid output. While this rigidity makes the metric a stringent and highly useful sanity check, it also revealed that none of the tested models could produce syntactically valid XML at inference time.

What makes this outcome especially significant is not just the zero-percent success rate—it's the consistency of failure even across a large test set of 12,500 examples. In many cases, the generated output appeared nearly correct to a human reader, with the appropriate tags and structure generally in place. However, the presence of stray tokens, inconsistent nesting, or partially formed tags was enough to render these outputs unusable when passed through an XML parser. These minor but critical errors suggest that the model struggled to internalize XML syntax as a rigid grammar, treating it instead as loosely styled text.

The results highlight a deeper issue, the inability of the models to self-regulate structural completeness and syntax adherence. A core requirement in document automation systems. The binary nature of the metric leaves no room for partial credit, meaning that small formatting deviations led to total failure in validation, even when the semantic content and intent of the output were otherwise accurate. This poses a fundamental challenge for generative models in structured output domains, where formal correctness is non-negotiable.

5.4.2.BLEU Scores

Model/Rouge	Rouge-1	Rouge-2	Rouge-L	Rouge-L Sum	BLEU
Base(Sheared LLaMa)	0.45	0.43	0.44	0.45	0.38
Finetune_50k	0.7	0.69	0.7	0.7	0.51
InstructTune_50k	0.66	0.64	0.65	0.65	0.28
InstructTune_100k	0.64	0.64	0.64	0.64	0.38

Figure 18 – Model Evaluation Metric Score Grid

Across all four test iterations, the highest BLEU score recorded was 0.51, achieved by the Finetune_50k model. This configuration, which relied solely on raw text/XML pairs without instructional prompting, consistently outperformed the other variants in terms of token-level precision and structural accuracy. The relatively strong performance of this model suggests that a straightforward mapping between input text and output XML, when presented without additional linguistic overhead, allowed the model to learn the task more effectively.

In contrast, the InstructTune_50k model produced the lowest BLEU score of the group, as can be seen in the Metric Score Grid of Figure 18. This outcome highlights a key limitation: the addition of instructional prompts not only failed to improve performance but may have actively hindered the model's ability to generate precise, token-aligned outputs. The inclusion of instructions increased sequence length and complexity, potentially diluting the signal of the core learning task and introducing noise during training.

Interestingly, scaling the dataset in the InstructTune_100k model led to some improvement, bringing its BLEU performance closer to that of the Finetune_50k baseline. However, even with double the training data, this instruction-tuned model did not surpass the simpler fine-tuned variant. This suggests that instruction tuning—while conceptually beneficial in general-purpose natural language generation—offered minimal advantage in this tightly scoped, structurally rigid task. The models performed best when trained directly on aligned input-output pairs, with minimal additional formatting or abstraction.

These results reinforce the notion that task-specific structure and format sensitivity outweigh the potential benefits of general instruction-following behaviour when generating highly constrained outputs like XML.

5.4.3.ROUGE Scores

Mirroring the trends observed in the BLEU score evaluation, the Finetune_50k model also outperformed all other configurations in terms of ROUGE scores, particularly ROUGE-L, which measures the longest common subsequence between the generated output and the reference XML. This metric is especially relevant in structured generation tasks, where preserving both the order and completeness of tags and content is crucial.

The strong ROUGE-L performance of the Finetuned model suggests that, among the tested configurations, it most consistently captured the overall structure and content alignment required for valid XML annotation. Unlike BLEU, which penalizes deviations at the n-gram level, ROUGE-L rewards continuity and sequence integrity, making it a valuable indicator of whether a model is on the right trajectory to produce syntactically and semantically coherent markup.

In contrast, both instruction-tuned models demonstrated weaker ROUGE performance. The InstructTune_50k model, in particular, lagged behind significantly—further reinforcing the observation that introducing an instructional prompt added complexity without a corresponding gain in fidelity. Even the InstructTune_100k model, despite its larger dataset, failed to outperform the simpler fine-tuned configuration.

Taken together, the ROUGE results underscore the relative advantage of direct, instruction-free training for tasks requiring rigid output formats. The high ROUGE-L score of the Finetuned model offers a promising signal: while the generated XML may still fall short of full well-formedness, the underlying structure is often largely intact, suggesting that with further refinement—such as stronger end-of-sequence modelling or tag-constrained decoding—it could produce valid and usable annotations.

5.5. Web App Results

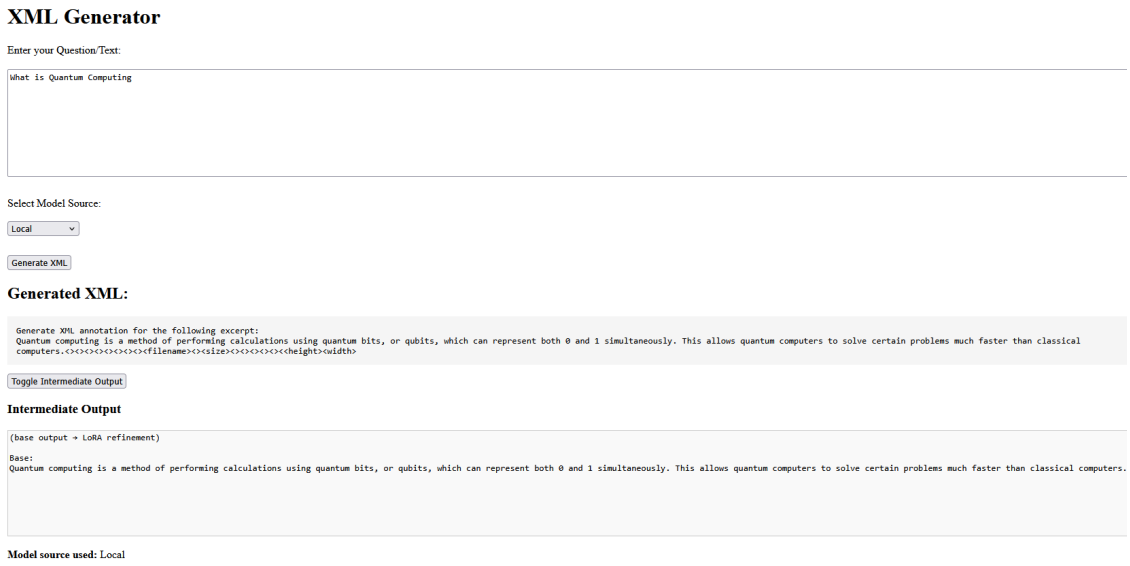


Figure 19 – Web App Interface with Real Model Output

With the training and evaluation of the models complete, the final phase of the project involved developing a Flask-based web application to serve as a practical interface for real-time document automation. This web app as seen in Figure 19, leverages the Sheared-LLaMA 1.3B model and integrates both the base model and its fine-tuned LoRA variant, allowing users to input text and receive XML-formatted outputs generated on the fly.

The application's core functionality is structured around a two-stage inference pipeline. When a user submits a query or excerpt through the interface—such as a passage from a document or a question—the system first formats this input into a standardized prompt. This prompt is then passed to the base Sheared-LLaMA model, which is loaded and executed locally on the server (either on a CUDA-enabled GPU or CPU, depending on availability).

In the local workflow, the base model processes the prompt and generates a plain-text response, intended to represent a coherent answer or interpretation of the user's input. This raw output is captured and immediately passed into a second prompt, prefixed with an instruction shown in Figure 20.

```
Generate XML annotation for the following excerpt:
{base_model_output}
```

Figure 20 – Prompt given to Finetuned Model with Base Model Response

This second prompt is handled by the LoRA-adapted model, which has been fine-tuned specifically for XML generation. The local model, now enhanced with task-specific LoRA layers, performs structured generation using beam search and token repetition controls to produce a valid XML snippet that mirrors the structure of the training annotations.

The result is then displayed on the webpage under two sections: one showing the raw intermediate output from the base model and another showing the final XML response generated by the LoRA model. This dual display provides transparency into the model's reasoning pipeline—effectively showing how an unstructured input is successively interpreted and refined into a structured output.

In scenarios where the local hardware is insufficient or model files are unavailable, the application offers an alternative route by leveraging Hugging Face's Inference API. In this case, the base model still runs locally to produce an initial interpretation, but the XML refinement step is offloaded to a hosted model through the API. The web app formats a payload and sends it to Hugging Face's server, receiving the XML response asynchronously, which is then rendered back to the user in the same interface.

Thus, the Flask app serves not only as a functional prototype but also as a demonstration platform for exploring structured generation workflows, model comparisons, and fallback routing between local and remote inference—all through an accessible and user-friendly web interface.

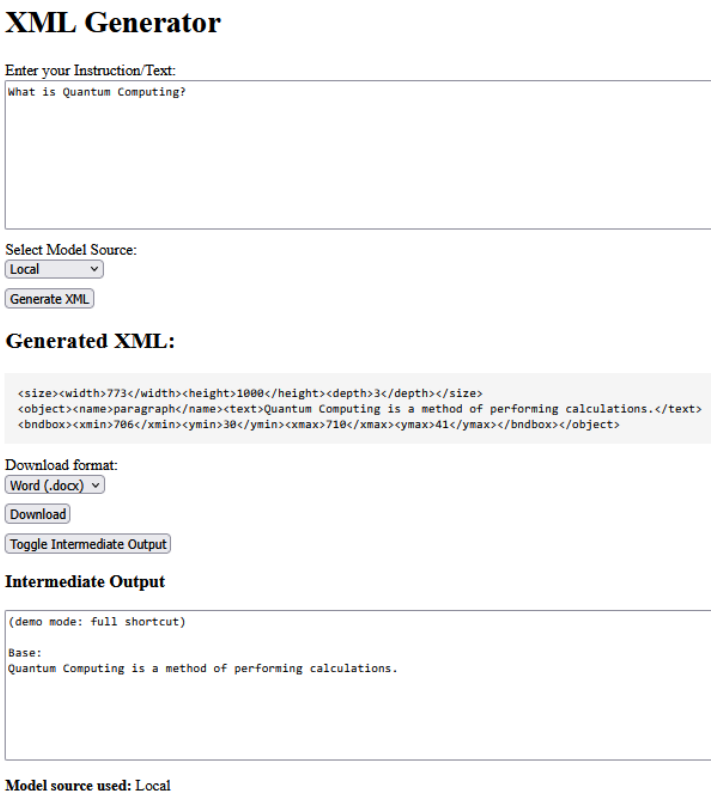


Figure 21 – Web App with exemplar model response

In reference to Figure 21, the intended full output within the webapp can be seen. The user asks a question or gives an instruction. As long as the generated XML is well formed, this can then be converted into a Word or PDF Document.

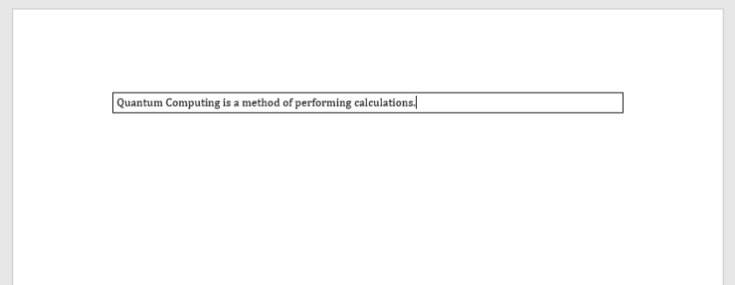


Figure 22 – Resulting Document of Converted XML

Figure 22 highlights the output of converting the above XML into a Word Doc. Completing the pipeline in full. It should be noted that this is to prove the proof of concept rather than creating the nested and multi-component XML which would be proceeded by Following the methodology with a configuration more fitting of complex XML such as training examples that feature more complex XML.

6. Conclusion

6.1. Final Remarks

The work undertaken in this project focused on developing, proposing and implementing a proof-of-concept framework tackling the problem space of Document Automation through the leveraging the advancements in Data Science and Artificial Intelligence. The resulting implementation shows a proof of concept that can be iterated and built upon by Amber Labs to develop a fully-fledged and realised framework by adhering to the methodology proposed.

The following is a discussion on how each objective has been met or approached:

- **Review existing literature and historical techniques in document automation and structured content generation.**
  This aim was met through the work presented within Section 2. This objective has been met through an historical exploration of previous techniques that have led towards the contemporary techniques within the current era of Machine Learning, which is heavily dominated by Large Language Models. Adjacent Work was not available to build upon, as highlighted within the section and as such the opportunity to explore the domain presented itself.

- **Develop a conceptual and technical blueprint for structured document generation.**
  This objective was fully addressed in the Methodology section, which proposed a complete end-to-end framework for structured document generation. The blueprint includes dataset preparation, prompt design, tokenizer usage, instruction formatting, fine-tuning setup with Sheared-LLaMA 1.3B, and LoRA-based adaptation strategies. Each step was supported with detailed reasoning (e.g., why <sep> was used as a structural delimiter) and aligned with state-of-the-art practices in large language model development, as described in Sections 4 and 5.

- **Identify and apply the proposed methodology to a representative problem space where document variation and structural integrity are both critical.**
  This objective was realized through the use of the DocBank dataset (as covered in the *Methodology* and *Results* sections), which presents a real-world challenge by combining rich layout annotations with raw text. The selection of DocBank ensured high document variation (12 distinct semantic labels) and the necessity for preserving strict XML

structure, satisfying the requirement for a problem space where both structural fidelity and variation are critical (see *Dataset Preparation* in Section 4.1).

- **Build a small-scale proof-of-concept system that demonstrates end-to-end transformation of raw or semi-structured input into well-formed, structured document outputs.**
  As detailed in Sections *Results* and *Web App Results*, a complete pipeline was implemented—from raw document text and annotations to tokenized training samples, LoRA fine-tuned model training, and a two-stage inference web interface. The Flask-based demo illustrated the real-time transformation of text into structured XML, with outputs displayed directly to users. While full XML well-formedness was not achieved during inference (see *XML Well-Formedness Scores*), the pipeline successfully demonstrated a functional proof-of-concept for automated structured document generation.

- **Define and apply both quantitative and qualitative metrics to evaluate output quality, accuracy, and structural validity.**
  Section 6 presents a comprehensive evaluation strategy, using BLEU, ROUGE-N, and ROUGE-L scores to quantitatively assess lexical precision, recall, and sequential alignment between generated and reference XML outputs. Additionally, XML well-formedness was used as a binary structural metric, implemented via xml.etree.ElementTree to validate syntactic correctness. These metrics were used to compare different model variants and generation settings, with key findings including the superiority of direct fine-tuning over instruction-based prompts and the impact of beam size and token limits on output quality (see *Validation Scores*, *GridSearch Optimisation*, and *BLEU/ROUGE Scores*).

- **Develop a lightweight inference application to showcase the framework in a user-facing environment.**

  A fully functional Flask web application was built and described in detail in the *Web App Results* section. It enabled users to input raw text and receive generated XML annotations using the fine-tuned Sheared-LLaMA 1.3B model with LoRA adaptation. The app also supported fallback to Hugging Face's Inference API for deployment in constrained environments. The interface transparently displayed both the raw model output and the XML result, completing the user-facing component of the framework.

6.2. Future Work and Recommendations

The following recommendations outline several promising directions in which this research could be extended. They are signposted as potential areas of interest for Amber Labs to explore in the development of a fully realised, production-grade document automation framework. Each recommendation builds on challenges and insights identified in this proof-of-concept and highlights opportunities to improve structural accuracy, output reliability, and model generalisation in future iterations.

Scale Training with Document-Level Context:
One of the most limiting factors in the current approach is the narrow training context, which is constrained to small excerpts of text rather than full documents. This restricts the model's ability to learn higher-order structural patterns and document-level dependencies such as ordering of sections, recurrence of layout types, or semantic hierarchies. Scaling up to include full document pages—or at least multi-paragraph spans—would allow the model to internalize global layout patterns and better understand how individual components fit within a broader structure. With the Sheared-LLaMA architecture's extended context support (up to several thousand tokens), this is a technically feasible enhancement that could significantly improve generation accuracy, particularly for multi-element or nested XML outputs.

Augment Training Data with Malformed and Corrected Examples:
To improve the model's ability to internalize the syntax of XML, one promising direction is the intentional inclusion of malformed examples during training—paired with their corrected equivalents. This technique mirrors approaches used in grammatical error correction and code auto-repair, where models learn to recognize and fix structural inconsistencies. By synthesizing examples where tags are left unclosed, misordered, or incorrectly nested, and explicitly demonstrating their proper form, the model may begin to generalize better rules around tag placement and closure. These corrective examples could also increase robustness by helping the model recognize subtle mistakes during generation and self-correct mid-sequence.

Explore Structured Prediction Models:
While the current methodology leverages autoregressive sequence-to-sequence generation, structured prediction models such as sequence-to-tree (seq2tree) or graph-based neural architectures may offer a more suitable framework for producing well-formed XML. These approaches naturally reflect the hierarchical and nested nature of markup languages, allowing the model to generate tags and content according to a structured, tree-like plan rather than a flat token sequence. A seq2tree model, for example, would explicitly construct parent-child relationships between nodes in the output, enforcing proper nesting and reducing the likelihood of malformed output. Similarly, graph neural networks could be used to encode inter-element relationships such as adjacency, containment, or dependency between layout blocks.

Incorporate Explicit End-of-Sequence Tokens for Tag Closure:
Another significant issue observed during evaluation was the model's inability to properly terminate XML tags or annotations, often leading to structurally incomplete outputs. To address this, future iterations could benefit from introducing explicit end-of-sequence tokens corresponding to the closure of key structural components and embedding these into the model's learning process as high-priority tokens. These tags could be emphasized in training through both position-aware loss weighting and through prompt scaffolding that consistently demonstrates their use. By treating these closures as structural delimiters

rather than generic tokens, the model could develop a more concrete understanding of document boundaries and syntactic rules.

Adopt Schema-Aware Fine-Tuning or Validation Feedback Loops
To ensure strict adherence to XML structure, another promising extension involves integrating schema-aware mechanisms during training and inference. This could be accomplished by validating model outputs against a defined XML Schema Definition (XSD) or Document Type Definition (DTD), either post-hoc or as part of a reinforcement learning framework where invalid outputs receive negative feedback. Alternatively, schema constraints could be encoded directly into the loss function or through masked decoding that prevents illegal tag sequences.

Bibliography

[1] E. Goldberg, “Using Natural-Language Processing to Produce Weather Forecasts,” 1994.

[2] E. Reiter and R. Dale, Building Natural Language Generation Systems, Cambridge University Press, 2000.

[3] I. Melcuk, “Meaning-Text Models: A Recent Trend in Soviet Linguistics,” *Ann. Rev. of Anthropology,* vol. 10, pp. 27-62, 1981.

[4] J. Coch, “Overview of AlethGen,” in *Eighth International Natural Language Generation Workshop*, 1996.

[5] K. McKeown, “Practical Issues in Automatic Documentation Generation,” 2002.

[6] A. Belz, “Automatic generation of weather forecast texts using comprehensive proabalistic generation-space models,” *Natural Language Engineering,* vol. 14, pp. 431-455, 2007.

[7] C. Thomson, “Comprehension Driven Document Planning in Natural Language Generation Systems,” 2020.

[8] L. Jianan, “LAYOUTGAN: GENERATING GRAPHIC LAYOUTS WITH WIREFRAME DISCRIMINATORS,” 2019.

[9] A. Radford, “Improving Laguange Understanding by Generative Pre-Training,” 2018.

[10] J. Devlin, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” 2019.

[11] S. Biswas, “DocSynthv2: A Practical Autoregressive Modeling for Document Generation,” Adobe Research, 2024.

[12] K. Deemter, “Template-Based Natural Language Generation: A Survey,” *Natural Language Engineering,* vol. 15, no. 3, p. 319–342, 2005.

[13] Z. Zhong, “PubLayNet: Largest Dataset Ever for Document Layout Analysis,” 2019.

[14] M. Li, “DocBank: A Benchmark Dataset for Document Layout Analysis,” 2020.

[15] BobLD, “PublayNetSharp,” 2023. [Online]. Available: https://github.com/BobLd/PublayNetSharp.

[16] A. Vaswani, “Attention Is All You Need,” *Advances in Neural Information Processing Systems,* p. 5998–6008, 2017.

[17] R. Liu, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” 2019.

[18] V. Sanh, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” 2019.

[19] C. Raffel, “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” 2019.

[20] C. Senadeera, “Controlled Text Generation using T5 based Encoder-Decoder Soft Prompt Tuning and Analysis of the Utility of Generated Text in AI,” 2022.

[21] A. Radford, “Language Models are Unsupervised Multitask Learners,” 2019.

[22] T. Brown and B. Mann, “Language Models are Few-Shot Learners,” 2019.

[23] OpenAI, “GPT-4 Technical Report,” 2023.

[24] L. Ouyang, “Training language models to follow instructions with human feedback,” 2022.

[25] Meta, “TheLlama3HerdofModels,” 2024.

[26] H. Touvron, “LLaMA: Open and Efficient Foundation Language Models,” 2023.

[27] H. Touvron, “Llama 2: OpenFoundation and Fine-Tuned Chat Models,” 2023.

[28] M. Xia, “Sheared LLaMa: Accerating Model pretraining via Structured Pruning,” ICLR, 2024.

[29] E. Hu, “LoRA: Low-Rank Adaption of Large Language Models,” 2021.

[30] P. Schmid, “Efficient Large Language Model training with LoRA and Hugging Face,” [Online]. Available: https://www.philschmid.de/fine-tune-flan-t5-peft. [Accessed 2025].

[31] T. Wolf, “Transformers: State-of-the-Art Natural Language Processing,” 2020.

[32] K. Papineni, “BLEU: a method for automatic evalution of Machine Translation,” in *ACL*, 2002.

[33] C.-Y. Lin, “ROGUE: A Package for Automatic Evasluation of Summaries,” in *ACL*, 2004.

[34] I. Sutskever, “Sequence to Sequence Learning with Neural Networks,” *Advances in Neural Information Processing Systems,* p. 3104–3112, 2014.

[35] W. Feng, “LayoutGPT: Compositional Visual Planning and Generation with Large Language Models,” Neural IPS, 2023.

[36] G. Kim, “OCR-free Document Understanding Transformer,” 2022.

[37] E. Hu, “LoRA: Low-Rank Adaption of Large Language Models,” 2020.

[38] R. Sennrich, “Neural Machine Translation of Rare words with subword units”.

Table of Figures

Figure 1 – Methodology Diagram 19
Figure 2 – Prepended Prompt 21
Figure 3 – Brevity Penalty Equation 26
Figure 4 – BLEU Score Equation 26
Figure 5 – ROUGE-N Equation 26
Figure 6 – ROUGE-L Equation 26
Figure 7 – Original JSON file of DocBank Annotations 27
Figure 8 – DocBank XML Annotation 28
Figure 9 – Contents of DocBank_txt folder 28
Figure 10 – Contents of Example Text document from folder 28
Figure 11 – Matched Text XML Pairs 29
Figure 12 – Instruction, Input, Output Triplets 29
Figure 13 – Decoded Tokenized Data 30
Figure 14 – Training Configuration Table 30
Figure 15 – Prompt Template 31
Figure 16 – Comparison of Model Loss across Epoch 32
Figure 17 – Grid Search Optimisation Search Space 33
Figure 18 – Model Evaluation Metric Score Grid 34
Figure 19 – Web App Interface with Real Model Output 36
Figure 20 – Prompt given to Finetuned Model with Base Model Response 36
Figure 21 – Web App with exemplar model response 37
Figure 22 – Resulting Document of Converted XML 38